PROJECT REPORT

Project Title : Multi-Client TCP Chat Application
Name : Ajay Kumar
Roll Number : 202310101200004
GitHub : https://github.com/ajay-navodayan/chat-app

1. OBJECTIVE

Design and implement a multi-client chat application using Python's socket and threading libraries, enabling real-time text communication between multiple users over a TCP network.

The project demonstrates core networking concepts including:

- TCP socket programming
- Client–server architecture
- Concurrent connection handling
- Custom application-layer protocol design

---

2. TECHNOLOGIES USED

Language: Python
Libraries: socket, threading, datetime
Protocol: TCP (Transmission Control Protocol)
Interface: Command-line (terminal-based)

---

3. SYSTEM ARCHITECTURE

The chat system follows a **Client–Server architecture**, where a central server manages communication between multiple connected clients

Server (server/server.py)

- Binds to host **0.0.0.0** on port **5000** and listens for incoming TCP connections.
- Spawns a dedicated thread for each connected client.

- Broadcasts messages to all clients.
- Handles clean and abrupt disconnections.

Client (client/client.py)

- Connects to the server at **127.0.0.1, port 5000**.
- Prompts the user for a username on startup.
- Spawns a background thread to receive incoming messages.
- The main thread reads user input and sends messages or commands.

Architecture Diagram

```
Client A
      \
Client B -----> TCP Server (Port 5000) -----> Broadcast Messages
      /
Client C
```

---

4. MESSAGE PROTOCOL

All communication uses a **simple text-based prefix protocol** transmitted over TCP. Each message begins with a keyword that identifies its type.

Prefix	Description
JOIN	Client registers with a chosen username
MSG	Client sends a chat message to be broadcast
/users	Client requests the list of online users
/quit	Client signals a graceful disconnect

/help To list all the commands

5. CONCURRENCY

Approach: Multi-threaded server (one thread per client)

The server uses Python's **threading module** to handle multiple clients simultaneously.

When a new client connects, a new thread is created to manage that client:

6. FEATURES IMPLEMENTED

Core Features

1. **Multi-client support**
   The server handles multiple concurrent client connections.
2. **Join and leave notifications**
   All clients receive a timestamped server message when a user joins or leaves.
3. **Broadcast messaging**
   Messages from one client are delivered to all other connected clients.
4. **Graceful disconnect**
   The **/quit** command disconnects the client and notifies others.
5. **Abrupt disconnect handling**
   Unexpected connection losses are detected and cleaned up.

Extra Features

6. **Message timestamps**
   Each message includes a timestamp
7. **User list command (/users)**
   Displays all connected users.
8. **Help command (/help)**
   Displays all available commands.

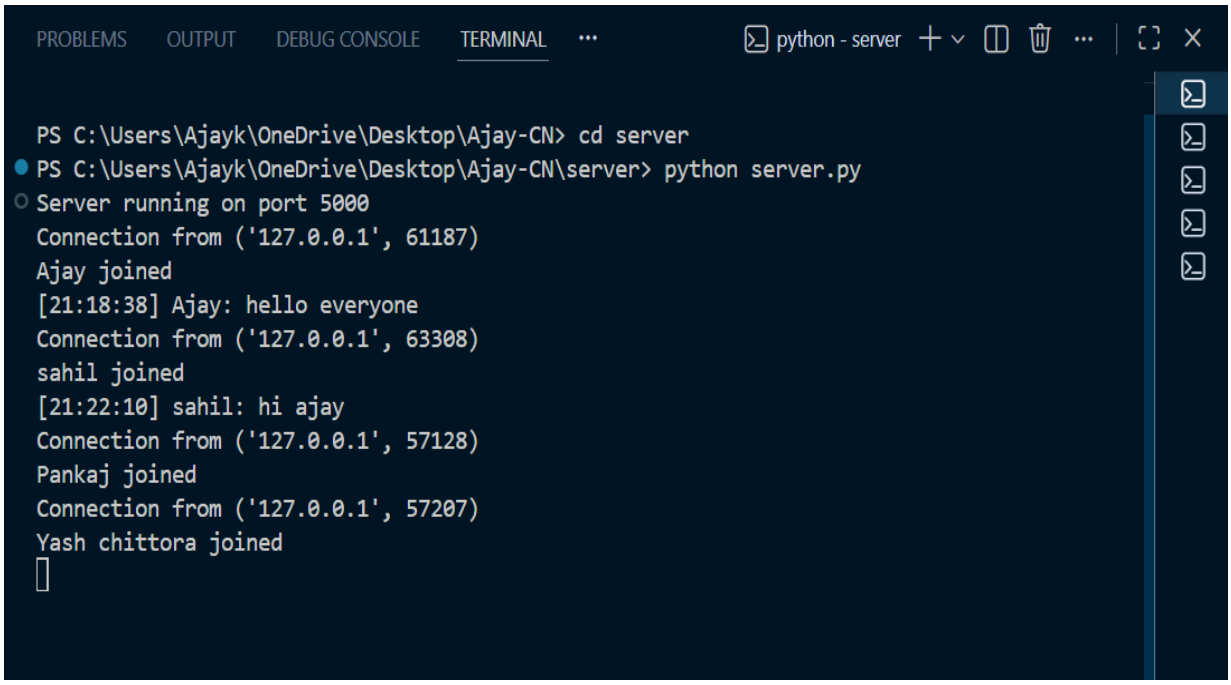
9. **Unknown command handling**
   Invalid commands show a helpful message.
10. **Message echo to sender**
    The sender receives their own message in the same formatted form.

7. TESTING

The application was tested using **multiple terminal windows** simulating several users.

Test — Testing Server and Multiple clients joining

Three client terminals connected successfully and received join notifications on the server.



Test — Message Broadcasting

To verify that messages sent by one client are successfully received by all other connected clients through the server.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    ···          python - client

adLine for compatibility purposes. If you want to re-enable it, run 'Import-Module PSReadL
ine'.

PS C:\Users\Ajayk\OneDrive\Desktop\Ajay-CN> cd client
PS C:\Users\Ajayk\OneDrive\Desktop\Ajay-CN\client> python client.py
Enter username: Ajay
[21:18:33] SERVER: Welcome to the chat!
hello everyone
[21:18:38] Ajay: hello everyone
[21:21:57] SERVER: sahil joined the chat.
[21:22:10] sahil: hi ajay
Enter username: Ajay
Enter username: Ajay
[21:18:33] SERVER: Welcome to the chat!
hello everyone
[21:18:38] Ajay: hello everyone
[21:21:57] SERVER: sahil joined the chat.
[21:22:10] sahil: hi ajay
[21:23:10] SERVER: Pankaj joined the chat.
[21:23:38] SERVER: Yash chittora joined the chat.
```

Test — Client disconnection (/quit)

One client exited using `/quit`. The server notified the remaining users.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    ···          python - server

Warning: PowerShell detected that you might be using a screen reader and has disabled PS
ReadLine for compatibility purposes. If you want to re-enable it, run 'Import-Module PSR
eadLine'.

PS C:\Users\Ajayk\OneDrive\Desktop\Ajay-CN> cd server
PS C:\Users\Ajayk\OneDrive\Desktop\Ajay-CN\server> python server.py
Server running on port 5000
Connection from ('127.0.0.1', 49848)
Ajay joined
[21:55:00] Ajay: Hi all
Connection from ('127.0.0.1', 64224)
Sahil joined
[21:55:17] Sahil: Hi ajay
[21:55:25] Sahil: how are you
[21:55:53] Ajay: i am good
Connection from ('127.0.0.1', 64340)
Harsh joined
Connection from ('127.0.0.1', 64395)
yash joined
[21:56:37] yash: hi harsh
Sahil disconnected
Harsh disconnected
```

Test — Server Stability with Multiple Connections

Multiple reconnections were performed and the server continued functioning correctly.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    ...    python - server

PS C:\Users\Ajayk\OneDrive\Desktop\Ajay-CN> cd server
PS C:\Users\Ajayk\OneDrive\Desktop\Ajay-CN\server> python server.py
Server running on port 5000
Connection from ('127.0.0.1', 49848)
Ajay joined
[21:55:00] Ajay: Hi all
Connection from ('127.0.0.1', 64224)
Sahil joined
[21:55:17] Sahil: Hi ajay
[21:55:25] Sahil: how are you
[21:55:53] Ajay: i am good
Connection from ('127.0.0.1', 64340)
Harsh joined
Connection from ('127.0.0.1', 64395)
yash joined
[21:56:37] yash: hi harsh
Sahil disconnected
Harsh disconnected
Connection from ('127.0.0.1', 51850)
[21:56:37] yash: hi harsh
Sahil disconnected
Harsh disconnected
Connection from ('127.0.0.1', 51850)
sahil joined
```

Test — Command Handling (/help and /users)

To verify that the chat application correctly handles built-in commands such as /help and /users

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    ...    python - client

Warning: PowerShell detected that you might be using a screen reader and has disabled PS
ReadLine for compatibility purposes. If you want to re-enable it, run 'Import-Module PSR
eadLine'.

PS C:\Users\Ajayk\OneDrive\Desktop\Ajay-CN> cd client
PS C:\Users\Ajayk\OneDrive\Desktop\Ajay-CN\client> python client.py
Enter username: Puneet
[22:10:20] SERVER: Welcome to the chat!
/h
Unknown command: /h  (type /help for available commands)
/help
Available commands:
  /help - Show all available commands
  /users - List all connected users
  /quit - Disconnect from the chat
/users
[22:10:43] SERVER: Online users (4): Ajay, yash, sahil, Puneet
```

9. CHALLENGES

Thread safety

Multiple threads access shared structures such as the clients list and usernames dictionary.
In production systems, **thread locks** should be used to avoid race conditions.

Sender message visibility

Initially, senders could not see their own messages.
This was solved by **echoing messages back to the sender**.

11. CONCLUSION

successfully implemented a **multi-client TCP chat application** using Python.

The application demonstrates key networking concepts including:

- TCP socket communication
- Client–server architecture
- Multi-threaded client handling
- Real-time message broadcasting

Additional features such as **timestamps, command support, and user lists** improve usability.

Future improvements may include:

- Private messaging
- GUI interface